Analytics Dashboard Manual Testing

1. Loading State

- **What expected (required) to see:** While fetching data from the API, the screen must display "Loading Metrics..." text, centered and styled in gray.
- **What we see:** The loading text appears correctly centered while the API request is in progress.

2. Access Restriction (Unauthorized/Forbidden)

- **What expected (required) to see:** In case of an error (401/403), show the "Restricted Access" header, a red `ShieldAlert` icon with an "animate-pulse" effect, and the corresponding error message.
- **What we see:** Correctly displays the restricted access UI with the pulsing red icon when the user is not authorized.

3. Page Header and Main Title

- **What expected (required) to see:** Page title "Platform Growth & Statistics" should be present with a `TrendingUp` icon in blue color on the left.
- **What we see:** The header is displayed clearly with the correct blue icon and appropriate font weight.

4. StatCard: General Design & Layout

- **What expected (required) to see:** Every card has a white background, "2rem" rounded corners, a subtle shadow, and a light gray border. Hovering over any card should increase the shadow (hover:shadow-md).
- **What we see:** All cards match the design specifications, and the hover effect works smoothly as expected.

5. StatCard: Content Alignment

- **What expected (required) to see:** Each card must have a color-coded square icon on the left, with the title (uppercase, gray, small font) and the numerical value (bold, large font) aligned on the right.
- **What we see:** Layout is consistent across all cards; icons and text blocks are aligned correctly.

6. Laundry Units Display

- **What expected (required) to see:** The "Laundry Units" card must correctly format the value as "{free_machines} Free / {total_machines}".
- **What we see:** The value is correctly formatted (e.g., "5 Free / 12") and matches the backend data.

7. Active Reports Conditional Styling

- **What expected (required) to see:** If `pending_reports > 0`, the card background must be `bg-rose-500` and it should have a visible `animate-pulse` effect. If 0, it should be a neutral `bg-slate-500`.
- **What we see:** The conditional styling triggers correctly based on the number of pending reports.

8. Icon Correctness

- **What expected (required) to see:** Verify that icons match the categories: Residents (Users), Signups (TrendingUp), Marketplace (ShoppingBag), Laundry (WashingMachine), Reports (AlertTriangle), Notifications (Bell).
- **What we see:** All icons are correctly associated with their respective metric categories.

Chat Component Manual Testing

1. Layout & Responsive Structure

- **What expected (required) to see:** Side panel is visible, with a dedicated area for search, specialized chats, and the conversation list. The right panel displays the active chat or a placeholder if none is selected.
- **What we see:** The interface renders perfectly with a clear separation between the sidebar and the active chat area.

2. Chat Search & User Addition

- **What expected (required) to see:** Typing an email and pressing "Enter" should find the user. Clicking the found user block should create a new conversation and navigate to it.
- **What we see:** Search functionality works correctly; new conversations are initiated and loaded as expected.

3. Specialized Chats (Global/Dorm)

- **What expected (required) to see:** Clicking "Global" or "Dorm" buttons should either navigate to an existing chat or create a new one if it doesn't exist.
- **What we see:** Both buttons correctly identify or create the necessary conversation types.

4. Optimistic UI (Message Sending)

- **What expected (required) to see:** Upon sending, the message should appear instantly in the chat with a `Clock` icon (isSending: true), and update to `CheckCheck` once confirmed by the server.
- **What we see:** Messages appear immediately; the transition from the sending state to the confirmed state is smooth and accurate.

5. Chat List Sorting & Pinning

- **What expected (required) to see:** Pinned chats should always stay at the top of the list. Pin toggling should work and persist in localStorage.
- **What we see:** The sorting logic functions correctly; pinning/unpinning updates the order instantly and persists after refresh.

6. Deleting Conversations

- **What expected (required) to see:** Clicking the trash icon should delete the chat, remove it from the list, and navigate away if the currently viewed chat was deleted.
- **What we see:** Deletion triggers the API call, updates the list, and handles the navigation gracefully.

7. Scroll Behavior

- **What expected (required) to see:** The chat window should automatically scroll to the latest message (`endRef`).
- **What we see:** The auto-scroll behaves reliably whenever `currentChat` updates.

8. Real-time Simulation

- **What expected (required) to see:** Background intervals should periodically fetch conversations and current chat data to keep the UI updated.
- **What we see:** The chat stays in sync with the server data without user intervention, as configured.

Dashboard Component Manual Testing

1. Initial Loading State

- **What expected (required) to see:** While fetching data from the API, a centered `Loader2` spinner should be displayed.
- **What we see:** The loading animation appears smoothly and vanishes immediately once data is loaded.

2. Header Banner & Personalization

- **What expected (required) to see:** Banner shows the correct user role (e.g., "admin" or "student"), a personalized greeting ("Hey, [Name]!"), and the `Zap` decorative icon in the corner.
- **What we see:** User role and name are correctly fetched from the profile API and displayed; the gradient styling is applied correctly.

3. Admin Analytics Section (Conditional Rendering)

- **What expected (required) to see:** This section is rendered only if `userRole === 'admin'`. The period dropdown (7 vs 30 days) should trigger a data refresh.
- **What we see:** Admin section is visible for admins and hidden for students. Period selection correctly triggers new API calls with the updated parameter.

4. Metric Cards (Admin only)

- **What expected (required) to see:** Cards for "Total Users", "Notifications", and "Marketplace" display correct values, icons, and color-coded backgrounds.
- **What we see:** All cards match the design; values reflect the latest backend data.

5. Quick Actions Navigation

- **What expected (required) to see:** "Laundry Room" and "Community Chat" cards must be clickable, leading to `/laundry` and `/chat` respectively. Hover effects (scale and border) should be active.
- **What we see:** Navigation links work perfectly; hover interactions are responsive and visually consistent.

6. Laundry Availability

- **What expected (required) to see:** "Laundry Room" card should show the current dynamic count of free machines.
- **What we see:** The count is accurately calculated from the machines list and updates dynamically.

7. Community Buzz Feed

- **What expected (required) to see:** Recent messages should be displayed in a 3-column grid. Anonymous/Empty messages should have a fallback display.
- **What we see:** The feed displays recent messages correctly, including sender initials and timestamps.

8. Error Handling & Data Sync

- **What expected (required) to see:** API failures should be caught without breaking the UI. Profile data should trigger a sync if the role in localStorage differs from the API response.
- **What we see:** The dashboard is robust; role synchronization happens correctly in the background, ensuring consistency across the app.

Events Component Manual Testing

1. Loading State

- **What expected (required) to see:** While the list of events is being fetched from the API, the text "Loading..." should appear.
- **What we see:** The loading state is displayed correctly and disappears once the event data is loaded.

2. Empty State

- **What expected (required) to see:** If there are no events in the system, the text "No events yet" should be visible.
- **What we see:** The empty state renders as expected when the event array is empty.

3. Create Event Modal

- **What expected (required) to see:** Clicking "Create" opens a modal with fields for Title, Description, Date (start/end), and Location. "Cancel" should close it, and "Create" should submit data.
- **What we see:** The modal opens and closes correctly; input fields are functional and accept the required data types.

4. Event Card Display

- **What expected (required) to see:** Each event card should clearly show the Title, Description (if provided), Start Date, End Date (if provided), Location, and the count of attendees.
- **What we see:** All event details are rendered correctly within the cards, following the provided design.

5. RSVP (Go/Leave) Functionality

- **What expected (required) to see:** Clicking "Go/Leave" should trigger the RSVP API, update the attendee count on the card, and show a success notification.
- **What we see:** The RSVP interaction works smoothly, updating the UI state immediately after the server responds.

6. Deleting Events

- **What expected (required) to see:** Clicking the trash icon should trigger the DELETE request, remove the event from the list, and show a success notification.
- **What we see:** The event is removed from the UI instantly upon successful deletion.

7. Date Formatting

- **What expected (required) to see:** Dates should be formatted into a readable local string (e.g., "Nov 25, 14:00").
- **What we see:** The `formatDate` helper correctly converts the ISO strings into a user-friendly format.

8. Form Validation/Submission

- **What expected (required) to see:** Form submission should send the data to the API and clear the form fields on success.
- **What we see:** The form submission creates the event correctly, and the modal state resets as expected.

ItemDetails Component Manual Testing

1. Data Fetching & Loading State

- **What expected (required) to see:** A loading spinner (`LoaderIcon`) should be displayed while data is being fetched. Upon completion, the item details should render, or an "Item not found" message should appear if the fetch fails or returns no data.
- **What we see:** The loading state is handled correctly, and data renders properly after a successful request.

2. Navigation & Layout

- **What expected (required) to see:** The "Back to Marketplace" button must correctly navigate the user back to the marketplace page.
- **What we see:** Navigation functions correctly according to `react-router-dom`.

3. Permissions & Ownership Logic

- **What expected (required) to see:** If the current user is the owner of the item (`isOwner`), the contact buttons should be replaced by an informative message: "You cannot contact yourself."
- **What we see:** The logic comparing `currentUserId` and `sellerId` works correctly, and access to contact buttons is restricted for the owner.

4. Reporting Functionality (Flag)

- **What expected (required) to see:** The flag button (for reporting) should only be visible to non-owners. Clicking it should redirect the user to `/report/product/${itemId}` if authorized, or to the login page if not.
- **What we see:** Token verification and redirection to the report page are implemented according to the technical requirements.

5. Communication (Contact & Exchange)

- **What expected (required) to see:**
  - "Contact Seller" and "Propose Exchange" buttons must initiate an API call to create a chat and automatically send an initial message.
  - Automatic redirection to the newly created chat.
- **What we see:** The `startConversation` method successfully creates a chat and sends the message. `try-catch` error handling prevents the interface from crashing in case of message sending failures.

6. UI/UX Elements

- **What expected (required) to see:** The product card must correctly display the image (handling the `BASE_URL`), category, status (New/Used), and price.

- **What we see:** Styled badges (category, status) and price formatting match the design specifications.

7. Error Handling

- **What expected (required) to see:** `toast` notifications should appear during loading errors or when attempting to contact a seller without an authentication token.
- **What we see:** The `sonner` notification system correctly informs the user about the status of operations.

Laundry Component Manual Testing

1. Data Loading & State Handling

- **What expected (required) to see:** While the machine list is being fetched, a `Loader2` spinner should be displayed. Once loaded, the machines should be grouped by type (washer/dryer).
- **What we see:** The loading state is managed correctly, and the filtering logic effectively separates the machines into their respective categories.

2. Real-time Timer Updates

- **What expected (required) to see:** For occupied machines, the timer (`formatTimeLeft`) should update every second to show the remaining time.
- **What we see:** The `useEffect` with the `setInterval` and `setTick` (state-based force rerender) correctly triggers UI updates for the timers.

3. Machine Management (Status Update)

- **What expected (required) to see:** Clicking "Manage" should open a modal. Setting the status to "occupied" should validate inputs (name required, minutes between 1 and 200). Successfully setting the status should update the UI and close the modal.
- **What we see:** Input validation is in place for `userName` and `timerMinutes`. The `handleAction` function correctly sends data to the API and triggers a re-fetch of the machine list.

4. Admin Functionality

- **What expected (required) to see:**
    - **Add Machine:** Admins should see a "+" button. The modal should allow setting a name and type (washer/dryer), and successfully creating a machine should append it to the list.
    - **Delete Machine:** Admins should see a trash icon on each machine. Clicking it should remove the machine from the list after API confirmation.
- **What we see:** Both `handleCreate` and `handleDelete` correctly utilize the `isAdmin` check, communicate with the API, and refresh the local state.

5. Error Handling & Feedback

- **What expected (required) to see:** Appropriate `toast` notifications for success (status updated, machine created) and failure (server errors, missing fields, validation errors).
- **What we see:** The component provides robust feedback for all user actions using the `react-hot-toast` library.

6. User Interface & Responsive Design

- **What expected (required) to see:** The list should be cleanly formatted, and the modal (Manage/Add) should overlay the screen effectively on both desktop and mobile viewports.
- **What we see:** The use of `fixed inset-0` and `backdrop-blur-sm` provides a professional modal experience that remains centered or bottom-aligned based on the screen size.

Login Component Manual Testing

1. Authentication Logic (Local)

- **What expected (required) to see:** Upon entering valid email/password, the user should be redirected to the homepage (`/`), and the tokens (`accessToken`, `refreshToken`) and user data should be correctly saved to `localStorage`.
- **What we see:** The `handleSubmit` function manages token storage and uses `updateRole` from `UserContext` to set the user state globally.

2. Google Authentication

- **What expected (required) to see:** Clicking "Continue with Google" should trigger the OAuth flow. Successful authentication should save data and redirect the user.
- **What we see:** The `useGoogleLogin` hook is implemented correctly. It includes a specific 403 error check to inform users if their Google account is not yet registered in the system.

3. Error Handling

- **What expected (required) to see:** Invalid credentials should display a red error message ("Invalid email or password.") and trigger a `toast.error`.
- **What we see:** Error states are reset before each attempt, and the UI provides clear feedback to the user if the server returns an error.

4. Loading States

- **What expected (required) to see:** When clicking "Sign In" or "Continue with Google", the button should become disabled, and a spinner (`Loader2`) should appear.
- **What we see:** The `loading` state correctly toggles during API requests, preventing multiple concurrent submissions.

5. Navigation

- **What expected (required) to see:** Links to "Forgot password?" and "Register" must correctly redirect the user to their respective paths.
- **What we see:** `react-router-dom` links are implemented correctly.

6. User Context/Role Update

- **What expected (required) to see:** After login, the application should know the user's role (e.g., 'admin' or 'student') to correctly gate access to other parts of the app.
- **What we see:** The logic fetches the user profile via `axios.get` after the token exchange and calls `updateRole` to update the React Context provider.

Marketplace Component Manual Testing

1. Data Loading & State Management

- **What expected (required) to see:** While fetching items, a `Loader2` spinner should be visible. Once loaded, all available items should be displayed in a 3-column grid.
- **What we see:** The `loading` state correctly handles the asynchronous API request, and the `formatted` array ensures that the data is structured correctly before being set to the state.

2. Search & Filter Functionality

- **What expected (required) to see:**
  - **Search:** The list should filter dynamically based on the input string (case-insensitive).
  - **Category Filters:** Selecting a category should display only items belonging to that category; selecting "Favorites" should show only items present in the `favorites` state.
- **What we see:** The `filtered` computation logic accurately combines both search criteria and category filtering.

3. Favorites System

- **What expected (required) to see:** Clicking the heart icon should toggle the favorite status (visual feedback) and trigger an API call to the server to persist the choice.
- **What we see:** The `toggleFavorite` function uses optimistic UI updates to keep the interface snappy, while the `axios` call handles the background synchronization.

4. Item Creation (Modal & Form)

- **What expected (required) to see:**
  - The modal opens via the "Add item" button.
  - Users can fill in the name, price, category, description, and "used" status.
  - Image upload with preview must work.
  - The `FormData` construction must include all fields correctly to match the Django backend expectations.
- **What we see:** The `handleSubmit` function uses `FormData` with the `multipart/form-data` header, which is correct for handling file uploads.

5. Error Handling & Validation

- **What expected (required) to see:** If the submission fails, the component should parse the backend error responses (e.g., validation errors) and display them in a `toast` notification.
- **What we see:** The `try-catch` block effectively maps the API error response to a user-friendly string.

6. User Interface & Responsiveness

- **What expected (required) to see:** The marketplace grid should be responsive, and the "Add item" modal should be properly centered with a backdrop blur effect.
- **What we see:** The layout uses Tailwind CSS classes for a modern, clean look that adheres to the overall design language.

Notifications Component Manual Testing

1. Data Loading & Background Refresh

- **What expected (required) to see:** Upon mounting, a loading spinner should appear. The component should poll the API every 30 seconds for new notifications without disrupting the UI (using the `silent` loading flag).
- **What we see:** The `loadNotifications` function correctly handles the initial state and background updates via `setInterval`.

2. Notification Filtering

- **What expected (required) to see:**
    - **Status Filter:** Toggling between "All" and "Unread" should instantly filter the displayed list.
    - **Type Filter:** Selecting a specific type (Message, Offer, Event, System) should correctly filter the list based on `notification_type`.
- **What we see:** The `filteredNotifications` logic successfully combines both filters to compute the display state dynamically.

3. Interaction & Read Status

- **What expected (required) to see:**
    - **Mark Single as Read:** Clicking a notification should trigger an API call to update its `is_read` status and immediately update the local UI (optimistic update).
    - **Mark All as Read:** The button should appear only if `unreadCount > 0` and should update all items at once.
- **What we see:** Both `handleMarkSingleAsRead` and `handleMarkAllAsRead` use optimistic updates, with fallback mechanisms to re-fetch the data if the server request fails.

4. Navigation Logic

- **What expected (required) to see:** Clicking a notification should redirect the user to the correct path based on `notification_type` (`/chat`, `/marketplace`, or `/events`) using the provided `target_id`.
- **What we see:** The `switch` statement inside `handleNotificationClick` handles the routing logic correctly for the supported types.

5. UI/UX Elements

- **What expected (required) to see:** * Unread notifications should be visually distinct (e.g., border styling).
    - The "unread count" badge should animate or appear correctly in the header.
    - Timestamps should be formatted human-readably (Today/Yesterday/Date).

- **What we see:** `formatTime` provides contextual date labeling, and Tailwind CSS provides clear conditional styling for read vs. unread states.

6. Empty States

- **What expected (required) to see:** If no notifications exist (or none match the filters), a clear empty state message should be displayed instead of a blank screen.
- **What we see:** The ternary operator check for `filteredNotifications.length === 0` handles this appropriately with a user-friendly message.

Profile Component Manual Testing

1. Data Loading & Authentication

- **What expected (required) to see:** Upon mounting, the component should fetch profile and product data. If no token exists, it must redirect to `/login`.
- **What we see:** The `useEffect` calls `fetchProfile`, which validates the existence of the `accessToken` and handles the request/loading states appropriately.

2. Role-Based Rendering

- **What expected (required) to see:** * "Room" information and inputs should be hidden for users with the role of `admin` or `doorkeeper`.
    - Dormitory and role information should be correctly displayed.
- **What we see:** The `isStaff` logic cleanly toggles the visibility of room-related UI elements in both the profile view and the edit modal.

3. Profile Editing & Image Upload

- **What expected (required) to see:**
    - The edit modal opens and pre-fills current data.
    - Changing the photo and profile details (name, dorm, room) should correctly send a `FormData` object to the server.
    - Upon success, the UI should update, and the new data should be persisted.
- **What we see:** The implementation uses `FormData` with `multipart/form-data`, ensuring compatibility with file uploads alongside text fields.

4. Listing Management (My Products)

- **What expected (required) to see:** * "Your Listings" should display the correct number of items.
    - Deletion should require a confirmation, and upon successful deletion, the item should be removed from the local state immediately.
- **What we see:** The `handleDeleteProduct` function includes a native `confirm` dialog and successfully updates the local `myProducts` state using `filter`.

5. Image Handling

- **What expected (required) to see:** Profile photos and product images should render correctly, with a fallback to `ui-avatars.com` if a user has no profile photo.
- **What we see:** The `getPhotoUrl` helper provides a robust way to handle both absolute URLs and relative paths from the backend.

6. User Experience & Responsiveness

- **What expected (required) to see:** The profile is responsive, showing a modern grid layout on large screens and stacking elements on smaller ones.
- **What we see:** Tailwind CSS grid and flexbox utilities ensure the layout adapts to screen size, and the edit modal uses a backdrop blur for focus.

Register Component Manual Testing

1. Role Selection & Conditional Fields

- **What expected (required) to see:** Selecting "Student" should show the "Room Number" input field. Selecting "Admin" or "Doorkeeper" should hide the "Room Number" field.
- **What we see:** The `selectedRole === 'student'` check in the JSX correctly manages the conditional rendering of the `roomNumber` input.

2. Form Validation Logic

- **What expected (required) to see:**
    - **Frontend Validation:** Empty fields should be caught before the API call.
    - **Password Complexity:** The requirements list (8+ chars, Uppercase, Number, Special) should update in real-time as the user types, using green checkmarks for satisfied conditions.
    - **Mismatch Check:** Passwords must match; if they don't, an error should be displayed.
- **What we see:** The `getPasswordRequirements` function and `validateForm` provide comprehensive client-side feedback.

3. API Integration & Error Handling

- **What expected (required) to see:**
    - The `handleSubmit` function should map Django-style error responses (like `UNIQUE constraint` on emails) to readable user messages.
    - Loading state should be active during the request to prevent double submissions.
    - Success should redirect to the `/login` page.
- **What we see:** The `try-catch` block effectively parses server errors, including special handling for specific database conflicts, and provides clear user feedback via `toast`.

4. UI/UX Elements

- **What expected (required) to see:**
    - "Back to Login" link works.
    - Inputs should have clear visual states (red borders/backgrounds) when errors occur.
    - Clearance of error states upon user re-entry into fields.
- **What we see:** The implementation utilizes conditional class names and `setErrors` management to clear feedback once a user corrects their input, which is a great UX practice.

ReportPage Component Manual Testing

1. Routing and State Initialization

- **What expected (required) to see:** The page should correctly extract `type` (e.g., 'product', 'user') and `id` from the URL via `useParams`.
- **What we see:** The component correctly uses `useParams` to define the target of the complaint, which is then used dynamically in the header text.

2. Form Submission and API Handling

- **What expected (required) to see:**
  - The form should prevent default submission behavior.
  - The `Authorization` header must include the `accessToken` from `localStorage`.
  - Successful submission should trigger a `toast.success` and navigate the user back to the previous page.
- **What we see:** The `handleSubmit` logic handles these steps. Note: Consider replacing `alert()` in the error block with a `toast.error()` for better UI consistency.

3. Interaction & Reason Selection

- **What expected (required) to see:** Users should be able to select a single reason (Spam, Fraud, etc.) from the list. The UI should provide immediate visual feedback (border/background changes) for the selected option.
- **What we see:** The `onClick` handler updates the `reason` state, and Tailwind conditional classes update the UI dynamically.

4. UI/UX Elements

- **What expected (required) to see:**
  - The design should be responsive and centered.
  - The Submit button must enter a `disabled` state while `loading` is true to prevent multiple API requests.
  - The "I changed my mind" button must successfully use `Maps(-1)`.
- **What we see:** The loading state is correctly tied to the button's `disabled` attribute, and the button text changes to "Sending...".

ReportDashboard Manual Testing

1. Data Fetching and State Management

- **What expected:** On mount, it should fetch the list of reports and display the total count and pending count.
- **What we see:** The `useEffect` triggers `fetchReports`, which correctly updates the state. The summary header uses `filter` to dynamically calculate the pending count.

2. Moderator Action Workflow

- **What expected:** Actions (`remove`, `warn`, `dismiss`) should be sent to the backend. Upon success, the UI should provide feedback and refresh the report list.
- **What we see:** The `handleAction` function correctly calls the backend, shows a toast success notification, and re-fetches the list, ensuring the "Handled" state is immediately reflected.

3. Conditional Rendering (Table & Empty States)

- **What expected:** The table should show actionable items for 'pending' status and a static "Handled" state for others. Empty states and loading indicators must display correctly.
- **What we see:** Ternary operators in the "Actions" cell and the bottom of the table effectively handle these UI states.

4. Visual Polish

- **What expected:** Consistent styling for status badges and moderator action buttons.
- **What we see:** `getStatusStyle` provides clear color-coding (Amber for pending, Emerald for resolved, Slate for dismissed), which is essential for a dashboard.

ResetPassword Component Manual Testing

1. URL Parameter Handling

- **What expected:** The component should extract the `token` from the URL via `useParams` to send to the backend.
- **What we see:** Correct usage of `const { token } = useParams<{ token: string }>();` ensures the API request will be linked to the correct reset session.

2. Password Strength UI

- **What expected:** Real-time feedback for the user regarding password requirements.
- **What we see:** The `passwordStrength` object tracks requirements, and the JSX updates the text color (`text-green-600` vs `text-gray-400`) based on the input status.

3. Form Logic & API Integration

- **What expected:** * Validation before submission (matching passwords and strength requirements).
    - Handling API errors (token expiration, validation errors).
    - Redirecting to `/login` after a successful update.
- **What we see:** The `handleSubmit` function performs necessary client-side checks and effectively handles server-side error responses (`err.response?.data?.detail`), providing appropriate user feedback.

4. Visual Polish

- **What expected:** User-friendly toggles (Show/Hide password) and clear loading states.
- **What we see:** The `showPassword` toggle is implemented correctly with the `Eye` icon. The success screen provides a clean transition away from the form.

VerifyEmail Component Manual Testing

1. URL Parsing & Initial State

- **What expected:** The component should automatically grab `token` and `email` from the query string upon mounting.
- **What we see:** The use of `useSearchParams` is correct, and the `useEffect` hook ensures the verification API call triggers immediately if a token exists.

2. API Interaction Logic

- **What expected:**
  - Success state: Display a success message and allow redirection to login.
  - Error state: Display the error message provided by the server and show a "Resend" button if the email parameter is present.
- **What we see:** The `verifyEmail` and `handleResendVerification` functions are correctly separated. Error handling includes a check for `error.response?.data?.error`, which is excellent for displaying specific backend validation messages.

3. UI/UX Feedback

- **What expected:** Users should never be left guessing if a process is ongoing.
- **What we see:** You are using `Loader2` with `animate-spin` during the loading phase and `resending` state during the retry phase. This is professional and gives the user confidence in the system.

4. Routing & Cleanup

- **What expected:** Clean navigation paths.
- **What we see:** The `Link` to `/login` is correctly positioned in both success and error states.